

职业转型执行手册

后端 → AI Infra / AI 算力工程师

12 个月转型计划

主线：大模型推理平台 × GPU 性能 × 分布式系统

适用画像

38 岁、在职、已有多年后端研发与通信/专网经验；每天可投入 2-3 小时。目标不是与算法博士竞争训练研究，而是成为“能让大模型稳定、低成本、高并发运行”的系统工程师。

时间投入	作品原则	求职节奏	预算原则
每周 15 h；每月约 60-65 h	学习 40% / 项目 60%	第 6 月试投；第 9 月定向； 第 12 月冲刺	按需租卡，全年建议 ¥3,500 -6,500

版本：2026-09 | 执行周期：连续 12 个月

先看结论：一年后你要证明什么

不是“学过很多”，而是“有证据能干活”

证明	一年后应拿出的证据	招聘方看到的能力
1. 生产型推理平台	Helm/容器化部署、网关、限流、观测、灰度、故障恢复	后端与平台工程能力可直接迁移
2. 专业 Benchmark	TTFT/TPOT/ITL/P99/吞吐/显存/GPU 利用率，含原始数据	懂推理负载，不只是会启动服务
3. 性能优化闭环	profile → 假设 → 修改 → A/B → 结论；端到端数字	具备 GPU 性能工程思维
4. 多 GPU / 系统实验	TP=2、NCCL、拓扑、NUMA/网络诊断	能向 AI 算力与集群方向下钻
5. 开源背书	vLLM/SGLang/PyTorch/K8s GPU 生态 issue/PR	代码质量和协作能力被第三方验证

12 个月能力演进

阶段	月份	职业标签	关键输出
A. 推理入门	1-2	Backend + LLM Inference Fundamentals	PyTorch 推理与 KV Cache 实验
B. 服务与平台	3-6	AI Infrastructure Engineer	Inference Platform V1 + Benchmark
C. GPU 性能	7-8	AI Infra / GPU Performance	CUDA/Triton + Nsight 优化案例
D. 分布式算力	9-10	AI Compute / Distributed Inference	NCCL 多卡 + NUMA/RDMA 诊断

E. 背书与求职	11-12	AI Infra & LLM Inference Engineer	上游 PR + Platform V2 + 定向简历
----------	-------	-----------------------------------	----------------------------

唯一主仓库策略

新建一个 GitHub 组织或主仓库 ai-infra-roadmap，按 01-12 编号沉淀实验；同时把第 6 月的 llm-inference-platform 独立成旗舰仓库。不要把 12 个月拆成一堆只有几次提交的小仓库。

每周固定节奏（适配每天 2-3 小时）

星期	投入	动作
周一	2 h	读官方资料 + 画概念图
周二	2.5 h	最小实验/复现
周三	2.5 h	项目编码
周四	2.5 h	项目编码 + 测试
周五	2 h	benchmark / profiling
周六	3-4 h	整合、README、技术文章
周日	30 min	复盘、下周拆解；其余休息

防焦虑规则

同时只学一个月主题；收藏但不立即学习的新资料统一进入 backlog。每周必须有一个“可见提交”，每月最后一个周末只做验收和复盘，不开新坑。

执行总表

每月只有一个主目标、一个核心项目、一个硬验收

月	主题	GitHub	月底交付	GPU/预算
M1	Python / PyTorch 与 GPU 基础	仓库 ai-infra-roadmap	1 个可复现实验目录 + 1 篇《后端工程师眼中的一次 LLM 推理》笔记	本地 CPU 为主；T4/16GB 约 8–12 h；月预算上限 ¥100
M2	Transformer 推理机制	02-transformer-inference-lab	可运行代码 + benchmark.csv + 2 张图 + 技术说明	T4/16GB 或 RTX 3090/24GB，约 12–18 h；¥100–200
M3	vLLM 单卡服务	03-vllm-serving	LLM 推理服务 V0.1 + 架构图 + 3 分钟演示视频	RTX 3090/4090 24GB 或 L4 24GB，约 20–25 h；¥200–350
M4	SGLang 与基准测试	04-serving-benchmark	《vLLM vs SGLang 推理性能报告》V1（求职核心证据）	同一张 24GB GPU 做 A/B，约 25–35 h；¥300–500
M5	生产化可观测性	05-observability	可观测性套件 + 故障演练记录 + dashboard 截图	只在集成与压测时租 20–25 h；¥200–350
M6	Kubernetes GPU 推理平台 V1	llm-inference-platform-v1	旗舰项目 V1 + 5–8 分钟演示 + 中文技术文章；开始试投	单卡 24GB 约 25–35 h；K8s 可用单节点 k3s/kind；¥300–500
M7	CUDA 基础	07-cuda-kernels	CUDA kernel lab + 性能表 + 《第一次定位 GPU kernel 瓶颈》	RTX 3090/4090 或 A10 24GB，约 20–30 h；¥250–450
M8	Triton + Nsight	08-gpu-optimization-case	真实性能优化案例：争取端到端 +10% 或明确证明瓶颈不在该 kernel	RTX 4090/A10/A100，约 25–35 h；¥350–700
M9	多 GPU 与 NCCL	09-multi-gpu-inference	多 GPU 实验报告 + 故障排查手册；第二轮求职	2×RTX 3090/4090 或 2×A10；约 20–30 h；¥500–900
M10	Linux / NUMA / 网络 / RDMA	10-systems-lab	《GPU 服务器拓扑与数据路径诊断手册》	GPU 10–15 h；两台普通云主机做网络实验；若无

执行优先：作品 > 课程，证据 > “熟悉”

				RDMA，用 Soft-RoCE/概念验证；¥250-500
M11	源码与开源贡献	11-open-source-notes + 上游贡献	源码导读 + issue 复现 + 上游 PR 链接	按 issue 需要 10-20 h；¥150-350
M12	综合平台 V2 + 求职冲刺	llm-inference-platform-v2	旗舰项目 V2 + 作品集主页 + 简历 + 10 分钟演示视频 + 求职复盘表	单卡/双卡集中验收 30-45 h；¥600-1200

总预算与租卡原则

- 优先租 NVIDIA CUDA 生态卡；前 6 个月 24GB 卡最实用，7B/小模型实验足够。
- 双卡只在第 9 月集中租，不长期空置；先写好脚本、数据与验收表，再开机。
- A100/H100 只在需要更稳定 profile、BF16/FP8 或较大模型时短租；不以“用过高端卡”为成果。
- 每次租卡先执行环境采集：GPU 型号/驱动/CUDA/PyTorch/框架版本/CPU/内存/拓扑；实验结束立即导出原始数据。
- 价格按市场动态变化；表中给的是预算上限，不是实时挂牌价。若公司有测试 GPU 或国产卡环境，优先使用并增加“同一 workload 跨硬件适配”案例。

第 1 个月 | Python / PyTorch 与 GPU 基础：

先把一次推理看懂

建议投入 60–65 h | GPU：本地 CPU 为主；T4/16GB 约 8–12 h；月预算上限 ¥100

本月学习边界

- Python 科学计算最小集：NumPy、张量 shape/stride、广播、dtype、autograd 只理解机制
- PyTorch：Tensor、Module、DataLoader、inference_mode、CPU/GPU 搬运、FP32/FP16/BF16
- GPU 基础：显存、计算单元、kernel、host/device、同步与异步
- LLM 请求链路：tokenize → embedding → Transformer blocks → logits → sampling

看什么资料（按顺序）

- PyTorch Learn the Basics：Tensor、Build Model、Autograd、Optimization（训练章节只做一次）
- Hugging Face LLM Course 第 1–2 章：模型、Tokenizer、Pipeline
- Jay Alammar 《The Illustrated Transformer》或同等可视化材料

GitHub 项目

仓库 ai-infra-roadmap：新增 01-pytorch-inference。实现 CPU/GPU 推理、dtype 切换、显存与耗时记录；README 解释一次请求的数据路径。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 能解释 tensor 的 shape、stride、dtype、device
- 能用 torch.inference_mode() 跑通 0.5B–1.5B 模型并记录峰值显存
- 能在白板上讲清 prefill 与逐 token decode 的区别（先定性即可）

月底硬交付

1 个可复现实验目录 + 1 篇《后端工程师眼中的一次 LLM 推理》笔记

第 2 个月 | Transformer 推理机制：理解性能从哪里来

建议投入 60–65 h | GPU：T4/16GB 或 RTX 3090/24GB，约 12–18 h；¥100–200

本月学习边界

- Self-Attention、Q/K/V、causal mask、RoPE、RMSNorm、MLP
- KV Cache 的形状、容量估算、生命周期；prefill 计算密集与 decode 内存访问特征
- Batch、sequence length、tokens/s、TTFT、TPOT、ITL、P95/P99
- 量化只理解 FP16/BF16/INT8/INT4 的质量、显存、速度取舍

看什么资料（按顺序）

- Hugging Face LLM Course 中 Transformer/Tokenizer 相关章节
- Andrej Karpathy nanoGPT / minGPT：只读 model.py 的前向路径
- vLLM 文档中的 PagedAttention、Automatic Prefix Caching 概念页

GitHub 项目

02-transformer-inference-lab：用 PyTorch 实现简化 causal attention + KV Cache；对比无缓存/有缓存 decode，输出延迟与显存曲线 CSV。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 能按层估算 7B FP16 权重约需多少显存，并说明还要预留什么
- 能用公式估算 KV Cache 随 batch、序列长度增长
- 实验能证明 KV Cache 降低重复计算，同时解释它为何增加显存压力

月底硬交付

可运行代码 + benchmark.csv + 2 张图 + 技术说明

第 3 个月 | vLLM 单卡服务：从模型脚本到在线系统

建议投入 60–65 h | GPU：RTX 3090/4090 24GB 或 L4 24GB，约 20–25 h；¥200–350

本月学习边界

- vLLM OpenAI-compatible server、SamplingParams、调度器基本概念
- Continuous Batching、PagedAttention、block/KV 管理、prefix caching
- Docker 镜像、模型缓存、健康检查、结构化日志
- 客户端并发、超时、重试、流式输出与背压

看什么资料（按顺序）

- vLLM Quickstart、Online Serving、Engine Arguments、Production Metrics
- vLLM 架构总览与 PagedAttention 设计文档
- OpenAI API 兼容协议相关示例

GitHub 项目

03-vllm-serving：FastAPI/Go 网关 + vLLM；支持鉴权、限流、超时、流式转发、request_id、健康检查；docker compose 一键启动。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 新机器按 README 在 30 分钟内启动服务
- 并发 1/8/32 三档压测无错误，保存配置与原始数据
- 能解释 continuous batching 与传统静态 batch 的差异

月底硬交付

LLM 推理服务 V0.1 + 架构图 + 3 分钟演示视频

第 4 个月 | SGLang 与基准测试：建立性能工程语言

建议投入 60–65 h | GPU：同一张 24GB GPU 做 A/B，约 25–35 h；¥300–500

本月学习边界

- SGLang server、Radix/Prefix Cache 基本思想、常用服务参数
- Benchmark 方法：固定模型、硬件、输入/输出长度、到达率、warm-up、重复次数
- TTFT、TPOT、ITL、E2E、request throughput、output tokens/s、显存与 GPU 利用率
- 公平对比与可重复性：版本锁定、随机种子、环境清单

看什么资料（按顺序）

- SGLang Benchmark and Profiling 官方文档
- vLLM Benchmark CLI 与 Production Metrics
- NVIDIA DCGM Exporter 指标说明（先会采集核心指标）

GitHub 项目

04-serving-benchmark：统一脚本测试 vLLM vs SGLang，生成 JSON/CSV 和图表；覆盖并发 1/8/32/64、短/长 prompt、prefix cache on/off。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 报告明确区分 TTFT、TPOT、吞吐和尾延迟
- 所有结论能由原始数据复现，不写“明显更快”之类空结论
- 能说明结果只对当前模型/版本/硬件/负载成立

月底硬交付

《vLLM vs SGLang 推理性能报告》V1（求职核心证据）

第 5 个月 | 生产化可观测性：把后端优势兑现

建议投入 60–65 h | GPU：只在集成与压测时租 20–25 h；¥200–350

本月学习边界

- Prometheus、Grafana、OpenTelemetry；RED/USE 方法
- 请求队列、running/waiting sequence、KV Cache 使用率、GPU util/显存/功耗
- 容量规划、过载保护、熔断、SLO、告警、日志与 trace 关联
- 负载模型：交互式聊天与批处理的不同目标

看什么资料（按顺序）

- vLLM Metrics / Prometheus and Grafana 示例
- Prometheus 官方入门与 Grafana dashboard 基础
- OpenTelemetry Python/Go instrumentation 官方文档

GitHub 项目

05-observability：为网关与 vLLM 建 dashboard；至少 12 个指标、5 条告警、1 条跨服务 trace；注入 OOM/进程退出/流量突增故障。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 5 分钟内从仪表盘定位一次人为故障
- 定义 2 个 SLO，并说明告警阈值与误报权衡

执行优先：作品 > 课程，证据 > “熟悉”

- 能从 QPS、token 长度、并发与 GPU 指标解释瓶颈

月底硬交付

可观测性套件 + 故障演练记录 + dashboard 截图

第 6 个月 | Kubernetes GPU 推理平台 V1: 形成第一版求职作品

建议投入 65–70 h | GPU：单卡 24GB 约 25–35 h；K8s 可用单节点 k3s/kind；¥300–500

本月学习边界

- K8s Deployment/Service/Ingress/ConfigMap/Secret/PVC、requests/limits
- NVIDIA device plugin、GPU resource、节点标签/污点/亲和性
- 滚动升级、灰度、HPA/KEDA 的边界；模型大文件冷启动
- Helm、CI、镜像安全和最小权限

看什么资料（按顺序）

- Kubernetes 官方 Concepts + Deployments + Scheduling
- NVIDIA k8s-device-plugin 官方 README
- vLLM Kubernetes/Helm 示例

GitHub 项目

llm-inference-platform-v1：Helm 部署网关+vLLM+监控；版本切换、限流、指标、故障恢复；GitHub Actions 做 lint/test/image build。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 一条命令安装，一条命令压测，一条命令卸载
- Pod 重启、模型服务故障后能恢复；操作有 runbook
- README 首页 60 秒内让招聘方看懂价值、架构、结果

月底硬交付

旗舰项目 V1 + 5-8 分钟演示 + 中文技术文章；开始试投

求职闸门

- 开始试投：LLMOps/MLOps 平台工程师、AI 平台后端、模型服务/推理平台研发、K8s GPU 平台研发、企业 AI 私有化部署工程师。
- 优先成都、远程、运营商/云厂商/AI 平台团队；也投内部 AI 算力、国产化适配、模型服务平台机会。
- 本阶段不硬投 CUDA Kernel/底层通信专家。目标不是马上离职，而是用 10-20 次面试校准缺口。

第 7 个月 | CUDA 基础：从“会用 GPU”到“能解释 GPU”

建议投入 60–65 h | GPU：RTX 3090/4090 或 A10 24GB，约 20–30 h；¥250–450

本月学习边界

- Thread/Block/Grid、Warp、SM、occupancy、kernel launch
- global/shared/register memory、coalescing、bank conflict、同步
- 向量加法、归约、矩阵乘法的朴素与 tiled 版本
- CUDA error checking、correctness、warm-up、event timing

看什么资料（按顺序）

- NVIDIA CUDA Programming Guide：Introduction、Programming Model、CUDA C++ Basics、Memory
- NVIDIA CUDA Samples：vectorAdd、reduction、matrixMul（理解，不照抄）
- PMPP（Programming Massively Parallel Processors）选读前 6 章

GitHub 项目

07-cuda-kernels：vector add、reduction、naive/tiled GEMM；单元测试对齐 PyTorch，benchmark 与 roofline 式定性分析。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 能解释 warp divergence、coalescing、shared memory 的作用
- 所有 kernel 有正确性测试且多尺寸覆盖
- 优化版相对朴素版有可测提升，并能说明为何提升/为何没提升

月底硬交付

CUDA kernel lab + 性能表 + 《第一次定位 GPU kernel 瓶颈》

第 8 个月 | Triton + Nsight：完成一次真实优化闭环

建议投入 60–65 h | GPU：RTX 4090/A10/A100，约 25–35 h；¥350–700

本月学习边界

- Triton program model、block pointer、mask、autotune
- 融合 kernel：softmax、layer norm 或 fused activation
- Nsight Systems 看 CPU/GPU timeline；Nsight Compute 看 kernel 指标
- memory-bound/compute-bound、launch overhead、同步点、数据搬运

看什么资料（按顺序）

- Triton 官方 Tutorials：Vector Add、Fused Softmax、Matmul、Layer Norm
- Nsight Systems User Guide：CUDA trace 与 NVTX
- Nsight Compute Profiling Guide 的核心指标章节

GitHub 项目

08-gpu-optimization-case：选一个推理相关算子，PyTorch eager → torch.compile/内置优化 → Triton；用 NVTX 和 Nsight 形成 before/after 证据。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 报告包含环境、方法、timeline/指标截图、正确性与性能
- 在至少 3 个 shape 上比较，避免只挑一个漂亮数字
- 能区分端到端收益和单 kernel microbenchmark 收益

月底硬交付

真实性能优化案例：争取端到端 +10% 或明确证明瓶颈不在该 kernel

第 9 个月 | 多 GPU 与 NCCL：跨过分布式推理门槛

建议投入 65–70 h | GPU：2×RTX 3090/4090 或 2×A10；约 20–30 h；¥500–900

本月学习边界

- collective：AllReduce、AllGather、ReduceScatter、Broadcast
- TP/PP/DP 的通信与显存取舍；单机 PCIe/NVLink 拓扑
- torch.distributed 与 NCCL 环境变量、错误定位
- vLLM tensor parallel；扩容为何可能变慢

看什么资料（按顺序）

- NVIDIA NCCL User Guide：collectives、usage、troubleshooting
- PyTorch Distributed Overview + NCCL backend
- vLLM Parallelism and Scaling / Distributed Troubleshooting

GitHub 项目

09-multi-gpu-inference：单卡 vs TP=2；采集吞吐、TTFT、TPOT、显存与通信 trace；运行 nccl-tests 并解释总线带宽。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 能画出 TP 一层内的主要通信点
- 能解释 2 卡为何不等于 2 倍速度
- 能用拓扑、消息大小、计算/通信比例解释实验结果

月底硬交付

多 GPU 实验报告 + 故障排查手册；第二轮求职

求职闸门

- 定向投递：AI Infra 工程师、LLM 推理工程师（平台偏工程）、GPU 云/算力平台研发、分布式推理工程师、AI 性能工程师（初中级）。
- 可投“熟悉 CUDA/NCCL 为加分项”岗位；对“精通 kernel/编译器/多年 RDMA”岗位仍谨慎。
- 用多卡实验与 Nsight 报告替代“熟悉”字眼，面试重点讲一次真实性能定位。

第 10 个月 | Linux / NUMA / 网络 / RDMA： 补齐算力工程底座

建议投入 60–65 h | GPU：GPU 10–15 h；两台普通云主机做网络实验；若无 RDMA，用 Soft-RoCE/概念验证；¥250–500

本月学习边界

- Linux perf、eBPF 基本定位；page cache、I/O、进程/线程、cgroup
- NUMA、CPU affinity、PCIe topology、pinned memory、H2D/D2H
- TCP vs RDMA；InfiniBand/RoCE、QP/CQ/MR、zero-copy、kernel bypass
- 跨机集合通信的带宽、时延、拥塞与 PFC/ECN 基本概念

看什么资料（按顺序）

- Linux perf 官方 wiki/man page；Brendan Gregg Linux Performance 方法论
- NVIDIA GPUDirect RDMA Overview
- RDMA Core / rdma-core examples 与 NCCL troubleshooting

GitHub 项目

10-systems-lab：lstopo/nvidia-smi topo/perf/iperf3/numactl 实验；可选 Soft-RoCE；写脚本自动采集系统拓扑与基准。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 能从 CPU、内存、PCIe、网络四层分析 GPU 喂不饱
- 能解释 NUMA 绑核/绑内存为何可能影响吞吐
- 没有真 RDMA 硬件时，明确区分已验证与理论理解

月底硬交付

《GPU 服务器拓扑与数据路径诊断手册》

第 11 个月 | 源码与开源贡献：获得第三方技术背书

建议投入 60–65 h | GPU：按 issue 需要 10–20 h；¥150–350

本月学习边界

- vLLM/SGLang 的入口、scheduler、worker/model runner、KV 管理、metrics
- 大型开源仓库的复现、最小测试、代码规范与 PR 沟通
- issue 分诊、benchmark 回归、文档与测试贡献路径
- 选择一个与你现有项目真实相关的问题

看什么资料（按顺序）

- 目标项目 CONTRIBUTING、architecture/design docs、good first issue
- 近 3 个月相关 PR：学习测试结构和维护者反馈方式
- GitHub Actions 与项目测试指南

GitHub 项目

11-open-source-notes + 上游贡献：至少提交 1 个有测试的 PR；优先 bug/test/benchmark/observability，不追求“大功能”。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 能沿一次请求画出关键调用路径，并标出 5 个核心文件
- 至少复现并记录 1 个真实 issue
- 目标：1 个 PR 合并；若未合并，至少得到 maintainer 有效 review 并持续迭代

月底硬交付

源码导读 + issue 复现 + 上游 PR 链接

第 12 个月 | 综合平台 V2 + 求职冲刺：完成身份迁移

建议投入 65–75 h | GPU：单卡/双卡集中验收 30–45 h；¥600–1200

本月学习边界

- 容量与成本模型、性能回归、发布与故障演练
- 多模型路由、优先级/配额、prefix cache、量化或 speculative decoding 选一
- 系统设计面试：推理网关、GPU 调度、跨机扩展、SLO
- STAR 项目表达与岗位定向简历

看什么资料（按顺序）

- 复盘全年官方资料；补齐目标 JD 高频缺口
- vLLM/SGLang 最新 release notes 与架构文档
- K8s/NCCL/CUDA 自制错题库

GitHub 项目

llm-inference-platform-v2：整合服务、K8s、观测、benchmark、多 GPU、成本模型、故障演练；发布 v1.0 release。

四周拆解

周	必须完成
W1	读核心资料；完成最小可运行样例；记录环境
W2	实现主功能；补测试；每次实验先固定变量
W3	压测/剖析；保存原始 JSON/CSV；形成第一版结论
W4	修正问题；完善 README/架构图/演示；按验收表打分

月底“过关”标准

- 陌生人按文档 45 分钟内跑通核心路径
- 至少 1 个有数字的优化结论，且实验可复现
- 能做 30 分钟架构讲解 + 30 分钟深挖问答
- 完成 30 家目标公司/团队清单、3 版定向简历、每周稳定投递

月底硬交付

旗舰项目 V2 + 作品集主页 + 简历 + 10 分钟演示视频 + 求职复盘表

求职闸门

- 冲刺：高级 AI Infra/模型推理平台工程师、AI 算力平台工程师、AI Systems Engineer、异构算力适配工程师、通信/专网 AI FDE/解决方案技术负责人。
- 对纯 CUDA Kernel/编译器岗位，只在已有高质量 kernel/上游贡献时投；对纯算法训练岗不作为主线。
- 目标组合：60% 高匹配、25% 向上挑战、15% 行业 AI/FDE。每周复盘简历转化率与面试缺口。

三个求职节点：能投什么、凭什么

节点	可投岗位	简历主证据	暂不主投
第 6 月	AI 平台后端、LLMOps/MLOps、模型服务平台、K8s GPU 平台、私有化部署	Platform V1、vLLM/SGLang benchmark、可观测性与故障演练	纯算法训练、CUDA Kernel 专家、RDMA 专家
第 9 月	AI Infra、推理平台、GPU 云平台、分布式推理、初中级性能工程	CUDA/Triton 优化、Nsight、TP=2/NCCL 报告	要求多年编译器/内核研发的岗位
第 12 月	高级 AI Infra、AI 算力平台、AI Systems、异构算力适配、行业 AI FDE	Platform V2、可复现数字、上游 PR、系统设计与行业经验	与个人证据不匹配的纯研究岗

针对你年龄与经历的定位

不要包装成 “AI 新人”

你的叙事应是：多年后端/分布式与通信专网经验，正在把生产系统能力迁移到 LLM 推理与 GPU 算力平台。38 岁不去与应届生拼 “会调 API”，而是证明能独立交付、定位复杂问题、把系统指标与业务成本连起来。

简历标题的阶段变化

时间	标题建议
现在	Backend / Distributed Systems Engineer 正在构建 LLM Inference Platform
M6	AI Infrastructure Engineer LLM Serving • Kubernetes • Observability
M9	AI Infra & GPU Performance Engineer vLLM • CUDA/Triton • NCCL

M12	AI Infra / AI Compute Engineer LLM Inference • GPU • Distributed Systems 通信/专网行业落地
-----	--

三类项目表达模板

- 平台类：为谁/什么 workload 构建什么平台 → 关键架构 → SLO/可靠性 → 验证结果。
- 性能类：基线 → profile 证据 → 瓶颈假设 → 改动 → 公平 A/B → 端到端收益与边界。
- 故障类：现象 → 指标/trace → 定位路径 → 根因 → 修复 → 防复发机制。

旗舰 GitHub 项目验收清单

招聘方打开仓库 60 秒，应知道你解决了什么

区域	必须有	优秀标准
首页	一句话价值、架构图、Quickstart、核心性能表	GIF/短视频；中英双语摘要
可复现	版本锁定、配置、数据生成、压测命令	Makefile/task runner 一键执行
工程质量	测试、CI、日志、错误处理、配置校验	安全扫描、release、changelog
性能证据	硬件/软件环境、原始数据、统计口径	参数 sweep、置信范围、回归阈值
生产能力	限流、超时、健康检查、metrics、告警	灰度、故障注入、容量/成本模型
深度	解释 scheduler/KV/cache/通信中的至少两项	源码调用链、上游 issue/PR

建议目录

llm-inference-platform/
 README.md # 价值、架构、Quickstart、结果
 docs/ # 设计、SLO、故障演练、源码导读
 gateway/ # Go 或 FastAPI 网关
 deploy/helm/ # K8s/Helm
 observability/ # dashboards、alerts、OTel
 benchmark/ # workloads、runner、raw、reports
 experiments/ # vLLM/SGLang、quant、prefix、TP
 scripts/ # env/topology/diagnostics
 tests/ # unit/integration/smoke
 .github/workflows/ # CI
 Makefile

最低数据表字段

类别	字段
环境	date, git_sha, model, engine_version, GPU, driver, CUDA, dtype
负载	input_len, output_len, concurrency, request_rate, duration, seed
延迟	TTFT p50/p95/p99, TPOT p50/p95/p99, E2E p50/p95/p99
吞吐	requests/s, input tokens/s, output tokens/s, total tokens/s
资源	GPU util, VRAM peak, power, CPU, RAM, KV cache usage
质量	success rate, errors, timeout, output consistency notes

验收评分表与调整机制

每月 100 分评分

维度	分值	判定
代码可运行	25	全新环境能按 README 跑通；有测试
数据可复现	20	原始数据、环境、命令、变量完整
理解可讲清	20	能脱稿讲 15 分钟并回答追问
工程完成度	20	错误处理、日志、配置、CI/部署具备
公开证据	15	README/报告/文章/演示至少两种

过关线

≥80 分进入下月；70-79 分用下一月第一周补齐；<70 分不得开启新框架。连续两月 <80 分，砍掉次要阅读，把项目范围缩小 30%，不增加每日时长。

每月复盘只回答 6 个问题

- 本月唯一硬交付是否公开、可运行？
- 我得到的一个可量化结论是什么？
- 哪个问题我现在能比月初讲清楚？
- 哪个知识点只是“看过”，还不能实验验证？
- 目标岗位 JD 新出现的 3 个高频词是什么？
- 下月要删掉什么，才能守住主线？

时间不够时的删减顺序

优先保留	其次保留	先删
旗舰平台代码与 README	官方文档核心章节	泛化课程、长视频

执行优先：作品 > 课程，证据 > “熟悉”

benchmark 原始数据与报告	必要的概念笔记	额外 Agent/RAG 框架
每月验收与演示	1 篇高质量文章	证书、重复搭环境

GPU 选择速查与实验纪律

场景	优先卡型	原因	不建议
M1-2 小模型/基础	T4 16GB; 3090/4090 24GB	便宜; 足够跑 0.5B-7B 的受控实验	一开始租 H100
M3-6 Serving/Benchmark	3090/4090 24GB; L4/A10 24GB	24GB 适合 7B、量化、服务压测	不同平台/卡型做横向结论
M7-8 CUDA/Profile	4090/A10; 短时 A100	较稳定地做 kernel 与 profile	只给 microbenchmark 不看 E2E
M9 多卡/NCCL	同型号 2×3090/4090/A10; 最好同机	减少硬件差异, 能研究拓扑/TP	混卡得出扩展效率结论
M10-12 综合	按目标 JD 选; 短时 A100/国产 NPU 可加分	对齐求职证据与异构适配	为“大模型参数量”烧预算

每次开 GPU 前的 15 分钟清单

- 代码、模型下载路径、数据集和命令已在本地准备; 先用 CPU/mock 冒烟。
- 设置自动关机/费用提醒; 估算本次最长时长和可接受失败次数。
- 保存 nvidia-smi、nvidia-smi topo -m、驱动/CUDA/PyTorch/框架版本。
- 先 warm-up, 再重复测量; 一次只改变一个关键变量。
- 实验结束导出 raw/、日志、profile 与图表; 确认落盘后关机。

全年预算建议

档位	年度预算	做法
节省版	¥2,500-3,500	大部分用 24GB 单卡; 双卡只做 1 次集中实验; 复用公司/本地环境
推荐版	¥3,500-6,500	按本计划执行; M8/M9/M12 给足实验时间

进阶版	¥7,000–12,000	增加 A100/多卡/真实 RDMA 或国产 NPU 适配，但不改变主线
-----	---------------	--------------------------------------

资料导航（以官方文档为主）

课程只服务项目，不以“看完”为目标

- [PyTorch Learn the Basics](#)
- [Hugging Face LLM Course](#)
- [vLLM Quickstart](#)
- [vLLM Documentation](#)
- [SGLang Benchmark and Profiling](#)
- [CUDA Programming Guide](#)
- [Nsight Systems User Guide](#)
- [NCCL User Guide](#)
- [Triton Tutorials](#)
- [Kubernetes Documentation](#)
- [NVIDIA k8s-device-plugin](#)
- [PyTorch Distributed Overview](#)
- [NVIDIA GPUDirect RDMA](#)
- [NCCL Tests](#)
- [Brendan Gregg Linux Performance](#)

资料使用规则

- 每个主题先读官方 Quickstart/Concepts，再通过项目遇到的问题回查深层章节。
- 博客和视频用于建立直觉；最终 README 的关键结论尽量链接官方文档、源码或论文。
- 框架更新快，执行到对应月份时检查最新版命令和 release notes；不要把版本号硬编码到职业认知里。

第一个 7 天：现在就开始

天	2-3 小时任务	当日可见结果
Day 1	建 ai-infra-roadmap；写目标、目录、12 月 milestone；配置 Python/uv 或 conda	首次 commit + 项目看板
Day 2	PyTorch tensor: shape/stride/dtype/device； 做 10 个小实验	notebook/script + 注释
Day 3	CPU vs GPU、同步计时、warm-up；记 录常见计时错误	timing.py + CSV
Day 4	加载 0.5B-1.5B 模型，跑 greedy/streaming；记录显存	infer.py + 环境记录
Day 5	画一次请求链路图；解释 tokenizer/prefill/decode	docs/request-path.md
Day 6	整理一键运行、错误处理、 requirements/lock；另一环境复现	README Quickstart
Day 7	15 分钟口述 + 复盘；列 5 个没讲清的问题	week-01-review.md

明晚睡前的最小交付

创建 GitHub 主仓库 ai-infra-roadmap，提交 README：写清转型标签、12 个月里程碑、每周 15 小时规则，并完成一个 PyTorch CPU/GPU tensor 示例。不要再搜索新路线。

最终提醒

这份计划的价值不在于“完整”，而在于连续 12 个月留下证据。第 6 个月就用市场反馈校准，第 9 个月用多 GPU 与性能案例提高技术标签，第 12 个月用平台 V2、开源贡献和通信/专网行业经验完成差异化。年龄不会因为多看一门课而消失，但会因为你有可复现、可量化、可讲清的系统成果而显著降低影响。